```
SELECT first_name, last_name, subsidiary_id, employee_id
  FROM employees
 WHERE ( subsidiary_id    = :sub_id OR :sub_id IS NULL )
   AND ( employee_id      = :emp_id OR :emp_id IS NULL )
   AND ( UPPER(last_name) = :name   OR :name   IS NULL )
```

The query uses named bind variables for better readability. All possible filter expressions are statically coded in the statement. Whenever a filter isn't needed, you just use `NULL` instead of a search term: it disables the condition via the `OR` logic.

It is a perfectly reasonable SQL statement. The use of `NULL` is even in line with its definition according to the three-valued logic of SQL. Nevertheless it is one of the *worst performance anti-patterns* of all.

The database cannot optimize the execution plan for a particular filter because any of them could be canceled out at runtime. The database needs to prepare for the worst case—if all filters are disabled:

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    2 |  478 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    2 |  478 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
1 - filter((:NAME   IS NULL OR UPPER("LAST_NAME")=:NAME)
       AND (:EMP_ID IS NULL OR "EMPLOYEE_ID"=:EMP_ID)
       AND (:SUB_ID IS NULL OR "SUBSIDIARY_ID"=:SUB_ID))
```

As a consequence, the database uses a full table scan *even if there is an index for each column*.

It is not that the database cannot resolve the "smart" logic. It creates the generic execution plan due to the use of bind parameters so it can be cached and re-used with other values later on. If we do not use bind parameters but write the actual values in the SQL statement, the optimizer selects the proper index for the active filter:

```
SELECT first_name, last_name, subsidiary_id, employee_id
  FROM employees
 WHERE( subsidiary_id    = NULL     OR NULL IS NULL )
   AND( employee_id      = NULL     OR NULL IS NULL )
   AND( UPPER(last_name) = 'WINAND' OR 'WINAND' IS NULL )
```

```
-------------------------------------------------------------------
|Id | Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |              |    1 |    2 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    2 |
|*2 |   INDEX RANGE SCAN           | EMP_UP_NAME  |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  2 - access(UPPER("LAST_NAME")='WINAND')
```

This, however, is no solution. It just proves that the database can resolve these conditions.



Warning

Using literal values makes your application vulnerable to SQL injection attacks and can cause performance problems due to increased optimization overhead.

The obvious solution for dynamic queries is dynamic SQL. According to the KISS principle[3], just tell the database what you need right now—and nothing else.

```
SELECT first_name, last_name, subsidiary_id, employee_id
  FROM employees
 WHERE UPPER(last_name) = :name
```

Note that the query uses a bind parameter.



Tip

Use dynamic SQL if you need dynamic **`where`** clauses.

Still use bind parameters when generating dynamic SQL—otherwise the "dynamic SQL is slow" myth comes true.

The problem described in this section is widespread. All databases that use a shared execution plan cache have a feature to cope with it—often introducing new problems and bugs.

[3] http://en.wikipedia.org/wiki/KISS_principle

MySQL

MySQL does not suffer from this particular problem because it has no execution plan cache at all . A feature request from 2009 discusses the impact of execution plan caching. It seems that MySQL's optimizer is simple enough so that execution plan caching does not pay off.

Oracle Database

The Oracle database uses a shared execution plan cache ("SQL area") and is fully exposed to the problem described in this section.

Oracle introduced the so-called *bind peeking* with release 9*i*. Bind peeking enables the optimizer to use the actual bind values of the first execution when preparing an execution plan. The problem with this approach is its nondeterministic behavior: the values from the first execution affect all executions. The execution plan can change whenever the database is restarted or, less predictably, the cached plan expires and the optimizer recreates it using different values the next time the statement is executed.

Release 11*g* introduced *adaptive cursor sharing* to further improve the situation. This feature allows the database to cache multiple execution plans for the same SQL statement. Further, the optimizer peeks the bind parameters and stores their estimated selectivity along with the execution plan. When the cache is subsequently accessed, the selectivity of the current bind values must fall within the selectivity ranges of a cached execution plan to be reused. Otherwise the optimizer creates a new execution plan and compares it against the already cached execution plans for this query. If there is already such an execution plan, the database replaces it with a new execution plan that also covers the selectivity estimates of the current bind values. If not, it caches a new execution plan variant for this query — along with the selectivity estimates, of course.

PostgreSQL

The PostgreSQL query plan cache works for open statements only—that is as long as you keep the `PreparedStatement` open. The above described problem occurs only when re-using a statement handle. Note that PostgresSQL's JDBC driver enables the cache after the fifth execution only.

SQL Server

SQL Server uses so-called *parameter sniffing*. Parameter sniffing enables the optimizer to use the actual bind values of the first execution during parsing. The problem with this approach is its nondeterministic behavior: the values from the first execution affect all executions. The execution plan can change whenever the database is restarted or, less predictably, the cached plan expires and the optimizer recreates it using different values the next time the statement is executed.

SQL Server 2005 added new query hints to gain more control over parameter sniffing and recompiling. The query hint `RECOMPILE` bypasses the plan cache for a selected statement. `OPTIMIZE FOR` allows the specification of actual parameter values that are used for optimization only. Finally, you can provide an entire execution plan with the `USE PLAN` hint.

The original implementation of the `OPTION(RECOMPILE)` hint had a bug so it did not consider all bind variables. The new implementation introduced with SQL Server 2008 had another bug, making the situation very confusing. Erland Sommarskog[4] has collected the all relevant information covering all SQL Server releases.

Although heuristic methods can improve the "smart logic" problem to a certain extent, they were actually built to deal with the problems of bind parameter in connection with column histograms and `LIKE` expressions.

The most reliable method for arriving at the best execution plan is to avoid unnecessary filters in the SQL statement.

[4] http://www.sommarskog.se/dyn-search-2008.html

Math

There is one more class of obfuscations that is smart and prevents proper index usage. Instead of using logic expressions it is using a calculation.

Consider the following statement. Can it use an index on NUMERIC_NUMBER?

```
SELECT numeric_number
  FROM table_name
 WHERE numeric_number - 1000 > ?
```

Similarly, can the following statement use an index on A and B—you choose the order?

```
SELECT a, b
  FROM table_name
 WHERE 3*a + 5 = b
```

Let's put these questions into a different perspective; if you were developing an SQL database, would you add an equation solver? Most database vendors just say "No!" and thus, neither of the two examples uses the index.

You can even use math to obfuscate a condition intentionally—as we did it previously for the full text LIKE search. It is enough to add zero, for example:

```
SELECT numeric_number
  FROM table_name
 WHERE numeric_number + 0 = ?
```

Nevertheless we can index these expressions with a function-based index if we use calculations in a smart way and transform the **where** clause like an equation:

```
SELECT a, b
  FROM table_name
 WHERE 3*a - b = -5
```

We just moved the table references to the one side and the constants to the other. We can then create a function-based index for the left hand side of the equation:

```
CREATE INDEX math ON table_name (3*a - b)
```

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 3

Performance and Scalability

This chapter is about performance and scalability of databases.

In this context, I am using the following definition for scalability:

> *Scalability is the ability of a system, network, or process,*
> *to handle a growing amount of work in a capable manner*
> *or*
> *its ability to be enlarged to accommodate that growth.*
>
> —Wikipedia[1]

You see that there are actually two definitions. The first one is about the effects of a growing load on a system and the second is about growing a system to handle more load.

The second definition enjoys much more popularity than the first one. Whenever somebody talks about scalability, it is almost always about using more hardware. *Scale-up* and *scale-out* are the respective keywords which were recently complemented by new buzzwords like *web-scale*.

Broadly speaking, scalability is about the performance impact of environmental changes. Hardware is just one environmental parameter that can change. This chapter covers other parameters like data volume and system load as well.

[1] http://en.wikipedia.org/wiki/Scalability

Performance Impacts of Data Volume

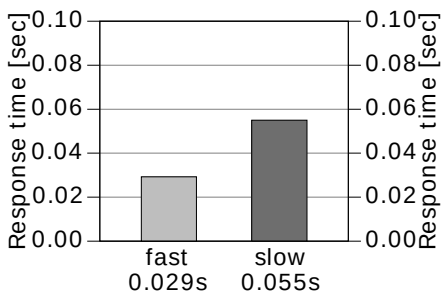
The amount of data stored in a database has a great impact on its performance. It is usually accepted that a query becomes slower with additional data in the database. But how great is the performance impact if the data volume doubles? And how can we improve this ratio? These are the key questions when discussing database scalability.

As an example we analyze the response time of the following query when using two different indexes. The index definitions will remain unknown for the time being—they will be revealed during the course of the discussion.

```
SELECT count(*)
  FROM scale_data
 WHERE section = ?
   AND id2 = ?
```

The column `SECTION` has a special purpose in this query: it controls the data volume. The bigger the `SECTION` number becomes, the more rows the query selects. Figure 3.1 shows the response time for a small `SECTION`.

Figure 3.1. Performance Comparison



There is a considerable performance difference between the two indexing variants. Both response times are still well below a tenth of a second so even the slower query is probably fast enough in most cases. However the performance chart shows only one test point. Discussing scalability means to look at the performance impact when changing environmental parameters—such as the data volume.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>



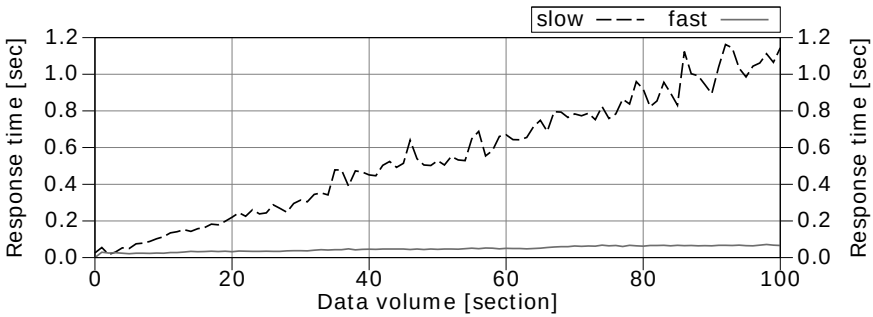
Important

Scalability shows the dependency of performance on factors like the data volume.

A performance value is just a single data point on a scalability chart.

Figure 3.2 shows the response time over the `SECTION` number—that means for a growing data volume.

Figure 3.2. Scalability by Data Volume



The chart shows a growing response time for both indexes. On the right hand side of the chart, when the data volume is a hundred times as high, the faster query needs more than twice as long as it originally did while the response time of the slower query increased by a factor of 20 to more than one second.

The response time of an SQL query depends on many factors. The data volume is one of them. If a query is fast enough under certain testing conditions, it does not mean it will be fast enough in production. That is especially the case in development environments that have only a fraction of the data of the production system.

It is, however, no surprise that the queries get slower when the data volume grows. But the striking gap between the two indexes is somewhat unexpected. What is the reason for the different growth rates?

It should be easy to find the reason by comparing both execution plans.

```
-----------------------------------------------------
| Id | Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |  972 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW |  3000 |  972 |
-----------------------------------------------------

-----------------------------------------------------
| Id   Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |   13 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST |  3000 |   13 |
-----------------------------------------------------
```

The execution plans are almost identical—they just use a different index. Even though the cost values reflect the speed difference, the reason is not visible in the execution plan.

It seems like we are facing a "slow index experience"; the query is slow although it uses an index. Nevertheless we do not believe in the myth of the "broken index" anymore. Instead, we remember the two ingredients that make an index lookup slow: (1) the table access, and (2) scanning a wide index range.

Neither execution plan shows a `TABLE ACCESS BY INDEX ROWID` operation so one execution plan must scan a wider index range than the other. So where does an execution plan show the scanned index range? In the predicate information of course!



Tip

Pay attention to the predicate information.

The predicate information is by no means an unnecessary detail you can omit as was done above. An execution plan without predicate information is incomplete. That means you cannot see the reason for the performance difference in the plans shown above. If we look at the complete execution plans, we can see the difference.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

```
-----------------------------------------------------
| Id | Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |  972 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW |  3000 |  972 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A))
       filter("ID2"=TO_NUMBER(:B))

-----------------------------------------------------
| Id   Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 |   13 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST |  3000 |   13 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A) AND "ID2"=TO_NUMBER(:B))
```



Note

The execution plan was simplified for clarity. The appendix on page 170 explains the details of the "Predicate Information" section in an Oracle execution plan.

The difference is obvious now: only the condition on `SECTION` is an access predicate when using the `SCALE_SLOW` index. The database reads all rows from the section and discards those not matching the filter predicate on `ID2`. The response time grows with the number of rows in the section. With the `SCALE_FAST` index, the database uses all conditions as access predicates. The response time grows with the number of selected rows.



Important

Filter predicates are like unexploded ordnance devices. They can explode at any time.

The last missing pieces in our puzzle are the index definitions. Can we reconstruct the index definitions from the execution plans?

The definition of the SACLE_SLOW index must start with the column SECTION—otherwise it could not be used as access predicate. The condition on ID2 is not an access predicate—so it cannot follow SECTION in the index definition. That means the SCALE_SLOW index must have minimally three columns where SECTION is the first and ID2 not the second. That is exactly how it is in the index definition used for this test:

```
CREATE INDEX scale_slow ON scale_data (section, id1, id2);
```

The database cannot use ID2 as access predicate due to column ID1 in the second position.

The definition of the SCALE_FAST index must have columns SECTION and ID2 in the first two positions because both are used for access predicates. We can nonetheless not say anything about their order. The index that was used for the test starts with the SECTION column and has the extra column ID1 in the third position:

```
CREATE INDEX scale_fast ON scale_data (section, id2, id1);
```

The column ID1 was just added so this index has the same size as SCALE_SLOW—otherwise you might get the impression the size causes the difference.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

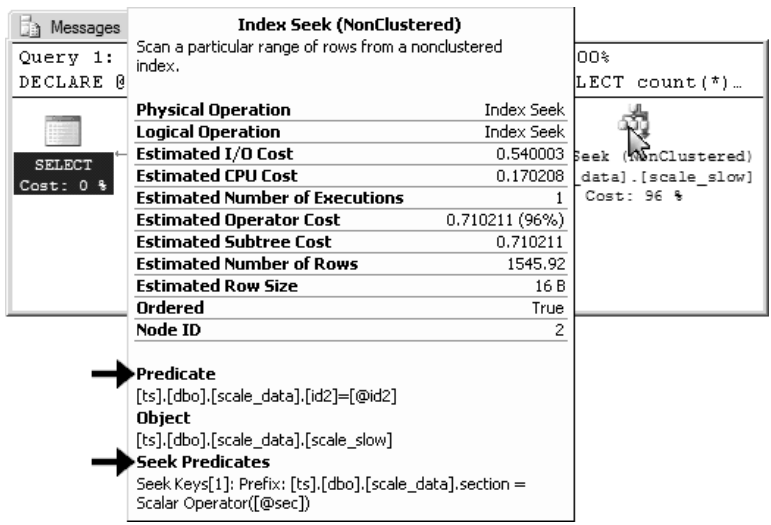
Performance Impacts of System Load

Consideration as to how to define a multi column index often stops as soon as the index is used for the query being tuned. However, the optimizer is not using an index because it is the "right" one for the query, rather because it is more efficient than a full table scan. That does not mean it is the optimal index for the query.

The previous example has shown the difficulties in recognizing incorrect column order in an execution plan. Very often the predicate information is well hidden so you have to search for it specifically to verify optimal index usage.

SQL Server Management Studio, for example, only shows the predicate information as a tool tip when moving the mouse cursor over the index operation ("hover"). The following execution plan uses the `SCALE_SLOW` index; it thus shows the condition on `ID2` as filter predicate (just "Predicate", without Seek).

Figure 3.3. Predicate Information as a Tool Tip

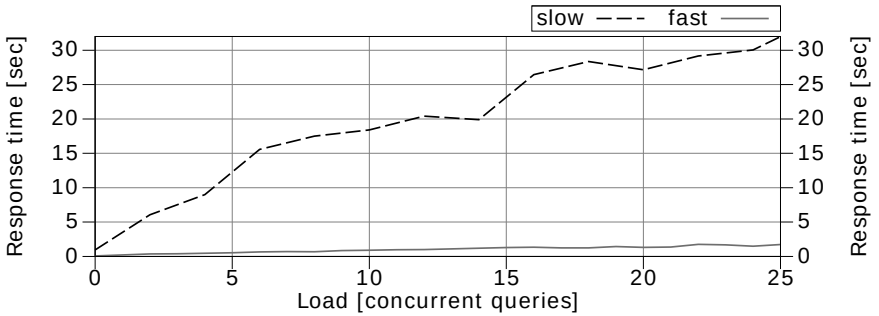


Obtaining the predicate information from a MySQL or PostgreSQL execution plan is even more awkward. Appendix A on page 165 has the details.

No matter how insignificant the predicate information appears in the execution plan, it has a great impact on performance—especially when the system grows. Remember that it is not only the data volume that grows but also the access rate. This is yet another parameter of the scalability function.

Figure 3.4 plots the response time as a function of the access rate—the data volume remains unchanged. It is showing the execution time of the same query as before and always uses the section with the greatest data volume. That means the last point from Figure 3.2 on page 81 corresponds with the first point in this chart.

Figure 3.4. Scalability by System Load



The dashed line plots the response time when using the SCALE_SLOW index. It grows by up to 32 seconds if there are 25 queries running at the same time. In comparison to the response time without background load—as it might be the case in your development environment—it takes 30 times as long. Even if you have a full copy of the production database in your development environment, the background load can still cause a query to run much slower in production.

The solid line shows the response time using the SCALE_FAST index—it does not have any filter predicates. The response time stays well below two seconds even if there are 25 queries running concurrently.



Note

Careful execution plan inspection yields more confidence than *superficial* benchmarks.

A full stress test is still worthwhile—but the costs are high.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Suspicious response times are often taken lightly during development. This is largely because we expect the "more powerful production hardware" to deliver better performance. More often than not it is the other way around because the production infrastructure is more complex and accumulates latencies that do not occur in the development environment. Even when testing on a production equivalent infrastructure, the background load can still cause different response times. In the next section we will see that it is in general not reasonable to expect faster responses from "bigger hardware".

Response Time and Throughput

Bigger hardware is not always faster—but it can usually handle more load. Bigger hardware is more like a wider highway than a faster car: you cannot drive faster—well, you are not allowed to—just because there are more lanes. That is the reason that more hardware does not automatically improve slow SQL queries.

We are not in the 1990s anymore. The computing power of single core CPUs was increasing rapidly at that time. Most response time issues disappeared on newer hardware—just because of the improved CPU. It was like new car models consistently going twice as fast as old models—every year! However, single core CPU power hit the wall during the first few years of the 21st century. There was almost no improvement on this axis anymore. To continue building ever more powerful CPUs, the vendors had to move to a multi-core strategy. Even though it allows multiple tasks to run concurrently, it does not improve performance if there is only one task. Performance has more than just one dimension.

Scaling horizontally (adding more servers) has similar limitations. Although more servers can process more requests, they do not the improve response time for one particular query. To make searching faster, you need an efficient search tree—even in non-relational systems like CouchDB and MongoDB.

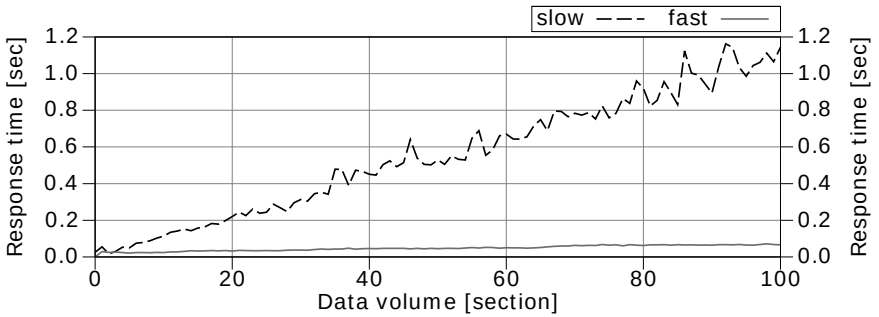


Important

Proper indexing is the best way to reduce query response time—in relational SQL databases as well as in non-relational systems.

Proper indexing aims to fully exploit the logarithmic scalability of the B-tree index. Unfortunately indexing is usually done in a very sloppy way. The chart in "Performance Impacts of Data Volume" makes the effect of sloppy indexing apparent.

Figure 3.5. Response Time by Data Volume



The response time difference between a sloppy and a proper index is stunning. It is hardly possible to compensate for this effect by adding more hardware. Even if you manage to cut the response time with hardware, it is still questionable if it is the best solution for this problem.

Many of the so-called NoSQL systems still claim so solve all performance problems with horizontal scalability. This scalability however is mostly limited to write operations and is accomplished with the so-called eventual consistency model. SQL databases use a strict consistency model that slows down write operations, but that does not necessarily imply bad throughput. Learn more about this in the box entitled "Eventual Consistency and the CAP Theorem".

More hardware will typically not improve response times. In fact, it might even make the system slower because the additional complexity might accumulate more latencies. Network latencies won't be a problem if the application and database run on the same computer, but this setup is rather uncommon in production environments where the database and application are usually installed in dedicated hardware. Security policies might even require a firewall between the application server and the database—often doubling the network latency. The more complex the infrastructure gets, the more latencies accumulate and the slower the responses become. This effect often leads to the counterintuitive observation that the expensive production hardware is slower than the cheap desktop PC environment that was used for development.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Eventual Consistency and the CAP Theorem

Maintaining strict consistency in a distributed system requires a synchronous coordination of all write operations between the nodes. This principle has two unpleasant side effects: (1) it adds latencies and increases response times; (2) it reduces the overall availability because multiple members must be available at the same time to complete a write operation.

A *distributed* SQL database is often confused with computer clusters that use a shared storage system or master-slave replication. In fact a *distributed* database is more like a web shop that is integrated with an ERP system—often two different products from different vendors. The consistency between both systems is still a desirable goal that is often achieved using the two-phase commit (2PC) protocol. This protocol established global transactions that deliver the well-known "all-or-nothing" behavior across multiple databases. Completing a global transaction is only possible if all contributing members are available. It thus reduces the overall availability.

The more nodes a distributed system has, the more troublesome strict consistency becomes. Maintaining strict consistency is almost impossible if the system has more than a few nodes. Dropping strict consistency, on the other hand, solves the availability problem and eliminates the increased response time. The basic idea is to reestablish the global consistency after completing the write operation on a subset of the nodes. This approach leaves just one problem unsolved: it is impossible to prevent conflicts if two nodes accept contradictory changes. Consistency is eventually reached by *handling* conflicts, not by *preventing* them. In that context, consistency means that all nodes have the same data—it is not necessarily the correct or best data.

Brewer's CAP Theorem describes the general dependencies between *C*onsistency, *A*vailability, and *P*artition tolerance.

Another very important latency is the disk seek time. Spinning hard disk drives (HDD) need a rather long time to place the mechanical parts so that the requested data can be read—typically a few milliseconds. This latency occurs four times when traversing a four level B-tree—in total: a few dozen milliseconds. Although that's half an eternity for computers, it is still far below out perception threshold…when done only once. However, it is very easy to trigger hundreds or even thousands disk seeks with a single SQL statement, in particular when combining multiple tables with a join operation. Even though caching reduces the problem dramatically and new technologies like SSD decrease the seek time by an order of magnitude, joins are still generally suspected of being slow. The next chapter will therefore explain how to use indexes for efficient table joins.

Solid State Disks (SSD) and Caching

Solid State Disks (SSD) are a mass storage technology that uses no moving parts. The typical seek time of SSDs is by an order of magnitude faster than the seek time of HDDs. SSDs became available for enterprise storage around 2010 but, due to their high cost and limited lifetime, are not commonly used for databases.

Databases do, however, cache frequently accessed data in the main memory. This is particularly useful for data that is needed for every index access—for example the index root nodes. The database might fully cache frequently used indexes so that an index lookup does not trigger a single disk seek.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 4

The Join Operation

An SQL query walks into a bar and sees two tables.
He walks up to them and asks 'Can I join you?'

—Source: Unknown

The join operation transforms data from a normalized model into a denormalized form that suits a specific processing purpose. Joining is particularly sensitive to disk seek latencies because it combines scattered data fragments. Proper indexing is again the best solution to reduce response times. The correct index however depends on which of the three common join algorithms is used for the query.

There is, however, one thing that is common to all join algorithms: they process only two tables at a time. A SQL query with more tables requires multiple steps: first building an intermediate result set by joining two tables, then joining the result with the next table and so forth.

Even though the join order has no impact on the final result, it still affects performance. The optimizer will therefore evaluate all possible join order permutations and select the best one. That means that just optimizing a complex statement might become a performance problem. The more tables to join, the more execution plan variants to evaluate—mathematically speaking: n! (factorial growth), though this is not a problem when using bind parameters.



Important

The more complex the statement the more important using bind parameters becomes.

Not using bind parameters is like recompiling a program every time.

Pipelining Intermediate Results

Although intermediate results explain the algorithm very well, it does not mean that the database has to materialize it. That would mean storing the intermediate result of the first join before starting the next one. Instead, databases use pipelining to reduce memory usage. That means that each row from the intermediate result is immediately *pipelined* to the next join operation—avoiding the need to store the intermediate result set.

Nested Loops

The nested loops join is the most fundamental join algorithm. It works like using two nested queries: the outer or driving query to fetch the results from one table and a second query *for each row* from the driving query to fetch the corresponding data from the other table.

You can actually use "nested selects" to implement the nested loops algorithm on your own. Nevertheless that is a troublesome approach because network latencies occur on top of disk latencies—making the overall response time even worse. "Nested selects" are still very common because it is easy to implement them without being aware of it. Object-relational mapping (ORM) tools are particularly "helpful" in this respect…to the extent that the so-called *N+1 selects problem* has gained a sad notoriety in the field.

The following examples show these "accidental nested select" joins produced with different ORM tools. The examples search for employees whose last name starts with `'WIN'` and fetches all `SALES` for these employees.

The ORMs don't generate SQL joins—instead they query the `SALES` table with nested selects. This effect is known as the "N+1 selects problem" or shorter the "N+1 problem" because it executes N+1 selects in total if the driving query returns N rows.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Java

The JPA example uses the CriteriaBuilder interface.

```
CriteriaBuilder queryBuilder = em.getCriteriaBuilder();
CriteriaQuery<Employees>
   query = queryBuilder.createQuery(Employees.class);
Root<Employees> r = query.from(Employees.class);
query.where(
  queryBuilder.like(
    queryBuilder.upper(r.get(Employees_.lastName)),
    "WIN%"
  )
);

List<Employees> emp = em.createQuery(query).getResultList();

for (Employees e: emp) {
  // process Employee
  for (Sales s: e.getSales()) {
    // process sale for Employee
  }
}
```

Hibernate JPA 3.6.0 generates N+1 **select** queries:

```
select employees0_.subsidiary_id as subsidiary1_0_
       -- MORE COLUMNS
  from employees employees0_
 where upper(employees0_.last_name) like ?

  select sales0_.subsidiary_id as subsidiary4_0_1_
         -- MORE COLUMNS
    from sales sales0_
   where sales0_.subsidiary_id=?
     and sales0_.employee_id=?

  select sales0_.subsidiary_id as subsidiary4_0_1_
         -- MORE COLUMNS
    from sales sales0_
   where sales0_.subsidiary_id=?
     and sales0_.employee_id=?
```